

LIVEQUE

Kickoff & Technical Specification

Version 1.0 — February 10, 2026

Developer: Isaac Irvin

Artist (Beta): Glen Irvin Jr.

Domain: getlivequeue.com

Current Version: v6.9.5

Production URL: <https://jolly-nougat-c7d3f5.netlify.app/>

LiveQue is a live music request platform that enables real-time interaction between performers and audiences. Customers scan QR codes to request songs with optional tips for priority placement, while performers manage their queue through an admin dashboard. This document covers the current state of the application, the technical architecture, and the complete roadmap to scale from a single-artist MVP to a multi-tenant SaaS platform.

Table of Contents

1. Executive Summary
2. Current State & Architecture
3. Database Schema
4. Application Pages
5. Top 3 Next Steps (with Time Estimates)
6. Payments Integration (Stripe + Fallback)
7. User Accounts & Authentication
8. Multi-Tenant Scalability Plan
9. Monetization: Free Tier & Subscription
10. Domain & Infrastructure Migration
11. End Gig & Session Data Capture
12. Full Feature Roadmap
13. Known Issues & Technical Debt
14. File Inventory
15. Development Principles

1. Executive Summary

1.1 What LiveQue Does

LiveQue replaces the awkward "shout your request at the musician" experience with a professional digital platform. Audiences scan a QR code at the venue, browse the artist's song library, and submit requests with optional tips for priority placement. The artist sees requests appear in real-time on their admin dashboard and manages their queue from their phone.

1.2 Key Features (Current — v6.9.5)

- Real-time song request queue with Supabase live sync
- Tip-based priority system with configurable tip amounts (\$0, \$2, \$5, \$10 default)
- 150+ song library including Spanish songs (flagged with country emoji)
- 2-hour cooldown system preventing repeat requests
- QR code generator with downloadable 4x6 PDF for venue table cards
- Admin dashboard with Live Stats, Quick Add, payment/social configuration
- Start New Gig flow that resets queue, cooldowns, and tips
- Toast notifications (replaced JavaScript alerts)
- Scroll-to-top and Feedback buttons
- Help modals on both customer and admin views
- Payment method links (Venmo, PayPal, CashApp, Zelle, Apple Pay)
- Social media profile links (Instagram, X, TikTok, Facebook, YouTube)

1.3 What We Are Building Next

The MVP is nearly complete for a single artist. The next phase transforms LiveQue from a single-artist tool into a scalable, monetized SaaS platform. This includes: real payment processing via Stripe, user accounts with authentication, multi-artist support, gig session tracking with analytics, a 3-free-gig trial model, and a \$7.99/month subscription tier.

2. Current State & Architecture

2.1 Technology Stack

Component	Technology	Notes
Frontend	Vanilla HTML/CSS/JavaScript	No framework; two separate HTML files
Backend / Database	Supabase (PostgreSQL + Realtime)	Real-time subscriptions + broadcast channels
Hosting	Netlify (static site)	Manual drag-and-drop deploy; migrating to getlivequeue.com
QR Code	qrcode.js + jsPDF	Client-side generation and PDF download
Payment Links	Direct URL redirects	Venmo, PayPal, CashApp, Zelle, Apple Pay
Domain	getlivequeue.com	Registered, no hosting configured yet

2.2 Live URLs

Admin Dashboard: <https://jolly-nougat-c7d3f5.netlify.app/>

Customer Page: <https://jolly-nougat-c7d3f5.netlify.app/customer.html>

2.3 Supabase Configuration

Project URL: <https://jttswydxqeyyqvcohnq.supabase.co>

Artist ID: f2c819fc-5665-4852-bcd7-797f92f7f0a5 (hardcoded; will become dynamic)

RLS: Currently DISABLED (must enable before public launch)

2.4 Application Architecture

The application consists of two HTML files that communicate through Supabase's real-time infrastructure. The admin dashboard (index.html) manages the queue, settings, and QR codes. The customer page (customer.html) displays the song library, current queue, and handles request submission. Both files subscribe to the same Supabase channels for instant updates across all connected devices.

Real-time sync uses two mechanisms: (1) Postgres Changes for database row inserts/updates/deletes, and (2) Broadcast channels for ephemeral events like "Start New Gig" clear signals. This dual approach ensures both persistent data and transient commands reach all clients immediately.

3. Database Schema

3.1 Current Tables

Table	Purpose	Key Columns
artist_settings	Per-artist configuration	artist_id, payment_methods, social_handles, tip_amounts, current_gig_session_id
songs	Song library	artist_id, title, artist_name, spanish (boolean)
requests	Active song request queue	artist_id, song_title, song_artist, requester_name, tip_amount, status, gig_session_id
played_songs	Recently played (for cooldowns)	artist_id, song_title, song_artist, played_at

3.2 Tables Needed for Scale

Table	Purpose	Key Columns
users	User accounts (Supabase Auth)	id, email, display_name, avatar_url, created_at
artist_profiles	Public artist info + slug	user_id, artist_name, slug, bio, profile_image
gigs	Gig session history	artist_id, session_id, started_at, ended_at, location, total_tips, total_requests, songs_played
subscriptions	Billing status	user_id, stripe_customer_id, stripe_subscription_id, status, plan, current_period_end
stripe_accounts	Stripe Connect accounts	user_id, stripe_account_id, onboarding_complete

4. Application Pages

4.1 Admin Dashboard (index.html) — 464 lines

Three tabs: Dashboard, Live Queue, and QR Code.

Dashboard Tab:

- Live Stats: request count, tip count (tap to reveal dollar amount), song library count
- Start New Gig: clears queue, resets cooldowns, broadcasts clear to customers, generates new session ID
- Payment Settings: configure Venmo, PayPal, CashApp, Zelle, Apple Pay handles
- Social Media: configure Instagram, X, TikTok, Facebook, YouTube handles
- Tip Options: set 4 tip amounts (first always \$0), saves to Supabase
- Song Library: add new songs with title + artist

Live Queue Tab:

- Quick Add grid: 4 random available songs with shuffle button
- View All Songs modal: searchable full library with availability status
- Now Playing card (green, pulsing) with large COMPLETE button
- Queued songs with Play and Done buttons; tipped songs highlighted in gold
- Clear Recently Played: resets all cooldown timers

QR Code Tab:

- Live preview of QR card with artist name
- Editable artist name field (updates card in real-time)
- Download for Print button: generates 4x6 inch PDF
- Link to Amazon for table card holders

4.2 Customer Page (customer.html) — 985 lines

- LiveQue branding with artist name, avatar initials, version badge, and LIVE NOW indicator
- Social media icon links (loaded dynamically from artist_settings)
- "Feeling Tipsy?" payment buttons (loaded dynamically; links open in new tab)
- Current Queue: real-time display of all requests; Now Playing in green, Up Next in teal
- Request a Song: searchable song library; tap to open request modal with tip options

- Request modal: shows song info, name input (optional), tip buttons from artist config
- Help modal: explains how requesting, queue, and cooldowns work
- Feedback button and scroll-to-top (appear on scroll-up)
- Real-time sync: requests appear/disappear instantly; Start New Gig clears customer view

4.3 Pages Needed for Scale

Page	Purpose
Landing page (getlivequeue.com)	Marketing page with signup CTA, feature overview, pricing
Signup / Login	Email+password and Google sign-in via Supabase Auth
Onboarding wizard	First-login flow: set name, add songs, configure payments, generate QR
Artist dashboard (/{{slug}}/admin)	Dynamic version of current index.html, loaded per logged-in user
Customer page (/{{slug}})	Dynamic version of current customer.html, loaded per artist slug
Subscription / Upgrade page	Pricing page shown after 3 free gigs; Stripe Checkout integration
Account settings	Profile, password change, billing management, Stripe Connect onboarding
Gig history / Analytics	Past gig data, charts, top songs, tip trends

5. Top 3 Next Steps (with Time Estimates)

These are ordered by dependency: each step builds on the previous. Completing all three gives you a functional multi-user MVP with real payments.

STEP 1: User Accounts & Authentication

4–6 hours

This is the foundation for everything else. Without user accounts, you cannot have multi-artist support, subscriptions, or per-user analytics.

What gets built:

- Supabase Auth integration with email+password and Google OAuth
- Signup page with email verification
- Login page with "Forgot Password" flow
- User profile creation on first signup (artist_profiles table with auto-generated slug)
- Protected admin routes: redirect to login if not authenticated
- Dynamic ARTIST_ID: replaced with logged-in user's ID throughout both files
- Row Level Security (RLS) policies on all tables
- Customer URL becomes getliveque.com/{artist-slug} (dynamic routing)

Why first: Stripe Connect requires a user account to attach to. Gig tracking requires knowing which user started the gig. Subscriptions require a user to bill. Everything depends on this.

STEP 2: Stripe Payments Integration

6–8 hours

Replace the current "link to Venmo" approach with real, in-app payments. Fans tap a tip amount and complete payment without leaving LiveQue. Money goes directly to the artist.

What gets built:

- Stripe Connect onboarding flow for artists (in account settings)
- Supabase Edge Function to create Stripe PaymentIntents
- Stripe Payment Element in the customer request modal (handles card, Apple Pay, Google Pay)
- Payment confirmation updates request with payment_id and confirms tip
- \$0 requests bypass Stripe entirely and submit directly
- Fallback mode: artists without Stripe still use Venmo/PayPal/CashApp links (current behavior)

- stripe_accounts table to track onboarding status
- Webhook endpoint to handle payment confirmations and failures

Key decision: Artists who don't want to set up Stripe keep the existing payment link experience as a fallback. This means free-tier users can still accept tips via Venmo etc., while Stripe becomes available for artists who want seamless in-app payments.

STEP 3: End Gig, Gig Tracking & Subscription Gate

5–7 hours

Close the loop on gig lifecycle, capture performance data, and gate access after 3 free gigs.

What gets built:

- "End Gig" button with confirmation modal ("Are you sure you want to end this session?")
- Gig summary capture on end: date, duration, location, total tips, total requests, songs played, top-tipped songs
- gigs table storing complete session history per artist
- Gig counter per user: tracks how many completed gigs
- On Start New Gig: check gig count; if ≥ 3 and no active subscription, redirect to upgrade page
- Upgrade page: clean, compelling design showing what you unlock; Stripe Checkout for \$7.99/mo
- Stripe Billing integration: create customer, manage subscription, handle webhooks
- subscriptions table tracking billing status per user
- Basic gig history view: list of past gigs with key stats
- A gig only counts toward the 3-free limit if at least 1 request was received (prevents accidental burns)

Total estimated time for all 3 steps: 15–21 hours

Each step is self-contained and deployable independently. Step 1 must come first; Steps 2 and 3 can run in parallel if needed.

6. Payments Integration (Stripe + Fallback)

6.1 Why Stripe Connect

Stripe Connect is designed for marketplace/platform models where a platform (LiveQue) facilitates payments between customers (fans) and sellers (artists). Each artist connects their own Stripe account and receives tips directly. LiveQue can optionally take a platform fee on each transaction in the future.

6.2 Payment Flow

1. Fan taps a song and selects a tip amount (e.g., \$5)
2. If artist has Stripe: customer modal shows Stripe Payment Element (card, Apple Pay, Google Pay)
3. If artist has no Stripe: customer modal shows existing payment link buttons (Venmo, PayPal, etc.)
4. For Stripe payments: frontend calls Supabase Edge Function to create PaymentIntent
5. Edge Function creates PaymentIntent with artist's Stripe Connect account as destination
6. Fan completes payment in the modal; Stripe confirms
7. On confirmation: request is submitted to queue with `payment_id` and verified tip amount
8. \$0 requests skip Stripe entirely and submit immediately

6.3 Stripe Connect Onboarding

Artists connect their Stripe account through an onboarding flow in their account settings. Stripe's hosted onboarding handles identity verification, bank account connection, and compliance. LiveQue stores the resulting `stripe_account_id` and `onboarding_complete` status. Artists who prefer not to onboard with Stripe continue using manual payment links.

6.4 Edge Function: create-payment-intent

A Supabase Edge Function handles PaymentIntent creation server-side (required for security; Stripe secret keys cannot be in client-side code). The function accepts the tip amount and artist ID, looks up the artist's Stripe Connect account, creates the PaymentIntent with the appropriate destination, and returns the `client_secret` to the frontend.

7. User Accounts & Authentication

7.1 Supabase Auth

Supabase provides built-in authentication with email+password and OAuth providers. We will enable email+password signup with email verification, and Google OAuth for one-click sign-in. Supabase handles password hashing, session tokens, refresh tokens, and password reset emails.

7.2 Signup Flow

1. User visits getlivequeue.com and clicks Sign Up
2. Chooses email+password or Google sign-in
3. For email: enters email, password; receives verification email; clicks link to confirm
4. For Google: redirected to Google consent screen; returns authenticated
5. On first login: onboarding wizard starts (set artist name, generate slug, add first songs)
6. `artist_profiles` row created with `user_id`, slug, and default settings
7. Redirected to their admin dashboard at getlivequeue.com/{slug}/admin

7.3 Route Protection

Admin pages check for a valid Supabase session on load. If no session exists, redirect to login. Customer pages (`/{{slug}}`) are always public — no auth required to request songs. API calls from the admin dashboard include the auth token; RLS policies verify the user owns the data they are accessing.

7.4 Row Level Security (RLS)

RLS is currently DISABLED. Before multi-user launch, every table must have policies ensuring artists can only read/write their own data. Example policy for the songs table: "Users can only SELECT/INSERT/UPDATE/DELETE rows where `artist_id = auth.uid()`". Customer-facing queries use the anon key and need SELECT-only policies scoped by `artist_id`.

8. Multi-Tenant Scalability Plan

8.1 The Core Change: Dynamic ARTIST_ID

Currently, ARTIST_ID is hardcoded as a constant in both index.html and customer.html. For multi-tenant, this becomes dynamic: the admin dashboard reads the logged-in user's ID from the Supabase session, and the customer page reads the artist slug from the URL path. Every Supabase query already filters by artist_id, so the migration is mostly about how that ID gets set rather than rewriting query logic.

8.2 URL Structure

URL Pattern	Page	How ARTIST_ID is Resolved
getliveque.com/	Landing / marketing page	N/A
getliveque.com/login	Login page	N/A
getliveque.com/signup	Signup page	N/A
getliveque.com/{slug}	Customer page (public)	Look up artist_profiles by slug
getliveque.com/{slug}/admin	Admin dashboard (protected)	auth.uid() from session
getliveque.com/account	Account settings (protected)	auth.uid() from session
getliveque.com/upgrade	Subscription page	auth.uid() from session

8.3 Routing on Netlify

Netlify supports client-side routing via a _redirects file. All /{slug} requests can be redirected to a single customer.html that reads the slug from window.location.pathname. Similarly, /{slug}/admin redirects to the admin dashboard. This avoids the need for a server-side framework while supporting clean, dynamic URLs.

8.4 Migration Checklist

- Replace hardcoded ARTIST_ID constant with dynamic resolution in both files
- Add artist_profiles table with slug field (unique, URL-safe)

-
- Create `_redirects` file for Netlify routing
 - Update all Supabase queries to use the dynamic ID (already parameterized, mostly a variable swap)
 - Add RLS policies to all tables
 - Update QR code generator to use `getliveque.com/{slug}` URL
 - Test with multiple artist accounts simultaneously

9. Monetization: Free Tier & Subscription

9.1 The Model: 3 Free Gigs, Then \$7.99/mo

New users get 3 free gigs to experience LiveQue with full functionality. After their 3rd completed gig, starting a 4th gig redirects them to the upgrade page. This lets artists prove the value at real performances before committing to a subscription.

9.2 What Counts as a Gig

A gig is only counted toward the free limit when the artist ends it AND at least 1 request was received during the session. This prevents accidental gig burns from testing or mis-clicks on Start New Gig. Empty sessions (no requests) are discarded and do not count.

9.3 Upgrade Page Design Philosophy

The upgrade page should feel like an opportunity, not a wall. Lead with: "You've had 3 great gigs on LiveQue" and show their actual stats (total requests, total tips earned). Frame the subscription as unlocking unlimited gigs plus future premium features. Include a clear CTA button that opens Stripe Checkout.

9.4 Pricing Considerations

- **Monthly:** \$7.99/month
- **Annual (future):** \$59.99/year (save ~37%) — consider adding after launch to increase commitment
- **Platform fee (future):** optional percentage on Stripe tips (e.g., 5%) as additional revenue stream
- **No free-tier feature limits for now:** all features available during the 3 free gigs so artists experience the full product

9.5 Stripe Billing Integration

Stripe Billing manages the subscription lifecycle: creating customers, charging monthly, handling failed payments, and managing cancellations. A Supabase Edge Function creates a Stripe Checkout Session, and a webhook updates the subscriptions table when payment status changes. The admin dashboard checks subscription status on load and on Start New Gig.

10. Domain & Infrastructure Migration

10.1 Domain Setup: getlivequeue.com

The domain is registered but has no hosting. The simplest path is to add it as a custom domain on the existing Netlify site. Netlify provides free SSL, CDN, and handles DNS once you point the domain's nameservers or add a CNAME record.

Steps to connect:

1. In Netlify dashboard: Site Settings > Domain Management > Add custom domain > getlivequeue.com
2. Netlify will provide DNS records (either nameservers or a CNAME)
3. At your domain registrar: update DNS to point to Netlify
4. Wait for DNS propagation (usually 15 minutes to 24 hours)
5. Netlify auto-provisions SSL certificate via Let's Encrypt
6. Update QR code target URL from Netlify subdomain to getlivequeue.com/{slug}
7. Update Supabase Auth redirect URLs to use getlivequeue.com

10.2 Email Service

Supabase Auth sends verification and password reset emails. By default these come from Supabase's domain. For production, configure a custom SMTP provider (Resend is a good choice since you are already using it for Beerlympics) so emails come from noreply@getlivequeue.com. This improves deliverability and brand trust.

10.3 Environment Variables

Variable	Where	Purpose
SUPABASE_URL	Frontend (public)	Supabase project URL
SUPABASE_ANON_KEY	Frontend (public)	Supabase anonymous key for client-side queries
STRIPE_PUBLISHABLE_KEY	Frontend (public)	Stripe public key for Payment Elements
STRIPE_SECRET_KEY	Edge Function (secret)	Stripe secret key for server-side API calls

Variable	Where	Purpose
STRIPE_WEBHOOK_SECRET	Edge Function (secret)	Verifies Stripe webhook signatures
RESEND_API_KEY	Edge Function (secret)	Email sending (if using Resend for transactional email)

11. End Gig & Session Data Capture

11.1 Current State

The "Start New Gig" button exists and works: it generates a new session ID, clears the queue, resets cooldowns, and broadcasts a clear signal to all connected customers. However, there is no "End Gig" button, and gig data is not saved anywhere. When Start New Gig is pressed, the previous session's data is effectively lost.

11.2 End Gig Flow

1. Artist taps "End Gig" button on the Dashboard tab
2. Confirmation modal: "Are you sure you want to end this gig? Your session data will be saved."
3. On confirm: system snapshots current session data
4. Data captured: gig_session_id, started_at (from session creation), ended_at (now), total requests, total tips (\$), songs played (list), top-tipped song, location (future: from artist input or geolocation)
5. Snapshot saved to gigs table
6. Queue is cleared, cooldowns reset, session ends
7. Toast: "Gig ended! Great set. Your stats have been saved."
8. Gig counter incremented for subscription gate logic

11.3 Gig Data for Analytics (Future)

Once gig data is being captured, a future analytics dashboard can show: gigs over time, average tips per gig, most-requested songs across all gigs, busiest days/times, tip trends, and comparative performance between venues. This data becomes a powerful retention tool: artists see the value LiveQue provides and are more likely to subscribe.

12. Full Feature Roadmap

12.1 Phase 1: MVP Completion (Current Focus)

- User accounts & authentication (email + Google)
- Stripe Connect for real tip payments
- End Gig + session data capture
- 3-free-gig trial + \$7.99/mo subscription
- Domain migration to getliveque.com
- Multi-tenant routing (/slug for customers, /slug/admin for artists)
- Row Level Security on all tables
- Onboarding wizard for new artists

12.2 Phase 2: Growth Features

- Gig analytics dashboard with charts (top songs, tip trends, gig history)
- Annual subscription option (\$59.99/yr)
- Custom domain email (noreply@getliveque.com)
- Artist profile pages with bio and photo
- Bulk song import (paste text list or upload CSV/TXT)
- Batch operations (clear multiple songs, bulk add to queue)

12.3 Phase 3: Premium Features

- Spotify integration (search catalog, auto-populate song details)
- Advanced cooldowns (customizable per-song, VIP bypass)
- SMS/push notifications for tips
- Live broadcast scoreboard (TV-ready display for venues)
- Platform fee on Stripe tips (optional revenue stream)
- Genre/mood filtering for song library
- Audience voting/poll features

12.4 Phase 4: Scale

- Native mobile apps (iOS + Android)
- Venue partnerships and enterprise tier

-
- API for third-party integrations
 - Multi-language support
 - White-label option for venues

13. Known Issues & Technical Debt

13.1 Critical (Must Fix Before Multi-User Launch)

- **RLS disabled:** All Supabase tables have Row Level Security turned off. Anyone with the anon key can read/write any artist's data. Must enable RLS with proper policies before adding users.
- **Hardcoded ARTIST_ID:** Both HTML files use a constant ARTIST_ID. Must become dynamic before multi-tenant.
- **No authentication:** Admin dashboard has no login. Anyone with the URL can manage Glen's queue.
- **Supabase anon key exposed in client code:** This is expected for Supabase (it's designed to be public), but RLS must be enabled to ensure it cannot be abused.

13.2 Important (Should Fix Soon)

- **Feedback button email obfuscation:** Uses Cloudflare's email-protection script which only works on Cloudflare-proxied domains. Will need updating when moving to getliveque.com.
- **No error logging service:** Errors only appear in browser console. Consider adding Sentry or LogRocket.
- **No automated testing:** All testing is manual. Consider adding basic integration tests.
- **Payment methods stored as JSON blob:** Works for now but may need normalization for analytics.

13.3 Development Constraints

- **JavaScript variable redeclaration:** Using let or const twice for the same variable name completely breaks the application. This is the #1 most common breaking error during development.
- **Inline styles:** Most HTML uses inline styles rather than CSS classes. Works but makes maintenance harder at scale.
- **No build step:** No bundler, minifier, or framework. Files are deployed as-is. This keeps things simple but limits code organization as files grow.

14. File Inventory

14.1 Current Files

File	Lines	Description
index.html	464	Admin dashboard: Live Queue, Dashboard settings, QR Code generator
customer.html	985	Customer-facing: song request, queue view, payment buttons, help

14.2 Files Needed for Scale

File	Purpose
landing.html	Marketing/home page for getlivequeue.com
login.html	Login page (email+password, Google OAuth)
signup.html	Signup page with email verification
onboarding.html	First-login wizard (name, songs, payments, QR)
account.html	Account settings, billing, Stripe Connect onboarding
upgrade.html	Subscription upgrade page with Stripe Checkout
analytics.html	Gig history and performance analytics
_redirects	Netlify routing rules for dynamic URLs
Edge: create-payment-intent	Stripe PaymentIntent creation (Supabase Edge Function)
Edge: create-checkout-session	Stripe subscription checkout (Supabase Edge Function)
Edge: stripe-webhook	Handle Stripe webhook events (Supabase Edge Function)

15. Development Principles

These principles have been established through trial and error during LiveQue's development and should be followed by any developer working on the project.

- **One feature at a time.** Never attempt multiple complex features simultaneously. Add one, test it, verify it works in production, then move to the next. This is non-negotiable.
- **Never redeclare variables.** Using `let` or `const` twice for the same name in the same scope breaks the entire application. Always check for existing declarations before adding new ones.
- **Talk before building.** Discuss the approach before writing code. Confirm the plan, then implement step by step.
- **Use existing working code as the base.** When debugging or adding features, start from the last known working version. Do not rebuild from scratch.
- **Test on device before deploying.** Every change should be verified in a browser before pushing to Netlify. Deploying costs money and broken deploys waste it.
- **Toast notifications, not alerts.** Never use JavaScript `alert()` dialogs. All user feedback should use the toast notification system.
- **Apple-inspired design.** Clean, minimal UI following iOS design patterns. Centered 400px container, subtle headers, proper visual hierarchy, no clutter.
- **Reliability is paramount.** Musicians depend on this during live performances. Any downtime during a gig is catastrophic. Err on the side of stability over features.

End of Document

LiveQue Kickoff Document v1.0 — February 10, 2026

Developer: Isaac Irvin | getlivequeue.com